INFO 5 – Projet 1 : Analyse mathématique et statistique de textes

Niveau : L1 Maths | Séance 2 sur 4 (3h)

Objectifs de la séance

- Implémenter un algorithme de tri classique (Tri par insertion).
- Modéliser mathématiquement des paires de lettres (les bigrammes).
- Manipuler une structure de données en 2 dimensions (matrice / liste de listes).
- Maîtriser l'utilisation des boucles imbriquées.

Partie 1 : Algorithmique de tri

Lors de la séance précédente, nous avons calculé les fréquences des lettres. Cependant, pour identifier la langue d'un texte, il est plus utile de classer ces lettres de la plus fréquente à la moins fréquente.

Exercice 1 : Le Tri par Insertion

Le tri par insertion consiste à parcourir un tableau et, pour chaque élément, à le faire "remonter" à sa place correcte parmi les éléments déjà triés à sa gauche. Pour ne pas perdre le lien entre une lettre et sa fréquence, nous allons trier une liste de couples `(lettre, frequence)`.

```python
def tri_insertion_frequences(frequences):
    """
    Trie les lettres par frequence decroissante.
    Renvoie une liste de couples (lettre, frequence).
    """
    # 1. Creation de la liste de couples
    donnees = []


    # 2. Algorithme du tri par insertion (ordre decroissant)
    for i in range(1, len(donnees)):



        # On decale vers la droite tant que l'element precedent a une frequence
        # plus petite que notre 'cle' (cle[1] correspond a la frequence)


        # On insere la cle a sa position definitive


    return donnees

# Question : Quelles sont les 5 lettres les plus frequentes de votre texte ?
```

Partie 2 : Modélisation matricielle des Bigrammes (2D)

Une analyse lettre par lettre n'est souvent pas suffisante. Nous allons analyser les paires de lettres consécutives, appelées **bigrammes** (ex : "th", "es", "le"). Mathématiquement, un bigramme peut être représenté dans une matrice M de taille 26×26. La case $M_{i,j}$ contient le nombre d'apparitions du bigramme commençant par la lettre d'indice i et finissant par la lettre d'indice j.

En Python, une matrice se représente par une **liste de listes**. Pour accéder à l'élément de la ligne i et de la colonne j, on écrira `matrice[i][j]`.

Exercice 2 : Remplissage de la matrice

```python
def compter_bigrammes(texte_propre):
    """
    Compte les bigrammes et les stocke dans une matrice 26x26.
    """
    # Initialisation securisee d'une matrice 26x26 remplie de zeros en Python
    matrice = [[0] * 26 for _ in range(26)]

    # On boucle jusqu'a l'avant-derniere lettre (pour lire des paires)


        # Calcul des indices de la ligne (i) et de la colonne (j)


        # Incrementation de la case correspondante dans la matrice


    return matrice
```

Partie 3 : Extraction et Synthèse

Maintenant que notre matrice est remplie, nous voulons extraire les bigrammes les plus fréquents pour les analyser. Il faut pour cela parcourir l'intégralité de notre matrice 26×26 à l'aide de **boucles imbriquées**.

Exercice 3 : Parcourir la matrice et afficher le Top 10

```python
def top_bigrammes(matrice, top=10):
    """
    Extrait les bigrammes de la matrice, les trie et affiche les plus frequents.
    """
    donnees = []

    # Double boucle pour parcourir les lignes (i) puis les colonnes (j)
    for i in range(____):
        for j in range(____):

            # Si le bigramme est apparu au moins une fois


                # Reconstruction de la chaine de 2 caracteres


                # Ajout du couple (bigramme, compte) a notre liste


    # Tri par insertion (identique a l'Exercice 1)


    # Affichage des 'top' premiers



# En francais, les bigrammes "es", "le", "de", "en" dominent generalement.
```